Finding Neighbours at Scale

Supplementary reading, lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

20 November 2026 · reading time about 25 minutes

Contents

Why a computer scientist should read this 1
1. The cost, stated precisely 2
2. The bargain: approximate answers 2
3. Three ideas, in increasing order of cleverness 2
    3.1 Hashing that preserves closeness 2
    3.2 Storing points approximately 3
    3.3 Navigable graphs 3
4. Where you will meet this 3
5. What to take from it 4
Where to look next 4

Not examinable. It is what happens to k-nearest neighbours when the dataset stops fitting in a notebook, and it is a large industry.

Why a computer scientist should read this

Lesson 6 dismissed k-nearest neighbours' cost in a sentence: $O(mn)$ per prediction, fine for 1,200 pumps, hopeless at scale.

That sentence hides a field. "Find the nearest points to this one" is the query behind product recommendation, duplicate detection, reverse image search, audio identification and plagiarism checking — systems answering thousands of such queries a second against hundreds of millions of stored items.

None of them can afford the loop you wrote in notebook 1. What they do instead is a good demonstration of an idea worth carrying: **when the exact answer is too expensive, change the question.**

This is also the part of lesson 6 that is most directly an engineering problem rather than a statistical one, which for this audience is a feature.

---

1. The cost, stated precisely

The brute-force search compares the query against every stored point:

- **Time:** $O(mn)$ for m stored items in n dimensions.
- **Memory:** the model *is* the dataset.

For a hundred million items in 128 dimensions, one query is about thirteen billion floating-point operations. At a thousand queries a second, that is not an optimisation problem; it is a different architecture.

And note what lesson 6 established about those dimensions: past a few dozen, the distances themselves stop discriminating. So the expensive computation is also, often, computing a quantity that has lost much of its meaning. Both problems have to be dealt with, and they pull in different directions.

---

2. The bargain: approximate answers

Here is the move that makes the field possible.

Exact nearest-neighbour search in high dimensions is essentially as expensive as scanning everything. The tree structures that work beautifully in two or three dimensions — k-d trees, ball trees — degrade towards a full scan as dimensions grow, for the same reason lesson 6 gave: when all distances are similar, no partition prunes much.

So production systems give up on *exact*. They answer:

> Return points that are **probably** among the nearest, quickly.

This is a genuine trade and it should feel uncomfortable at first. You are accepting a recall of, say, 95% — meaning one in twenty of the true nearest neighbours is missed — in exchange for two or three orders of magnitude in speed.

Why that is usually the right trade is worth stating, because it is not obvious. The application rarely needs the true nearest neighbour. A recommender suggesting the fourth-most-similar product instead of the first is not noticeably worse; a duplicate detector that misses one near-duplicate in twenty still removes 95% of them. The exactness was never the point — it was a property of the algorithm, not a requirement of the task.

Recognising when precision you are paying for is not precision you need is a professional skill well beyond this topic.

---

3. Three ideas, in increasing order of cleverness

3.1 Hashing that preserves closeness

An ordinary hash function scatters similar inputs to unrelated buckets — that is what makes it a good hash. **Locality-sensitive hashing** deliberately does the opposite: it is built so that nearby points collide *often* and distant points collide *rarely*.

Then the search is: hash the query, look only in its bucket. A scan of a few hundred candidates replaces a scan of a hundred million.

A concrete instance, for cosine similarity: pick a random direction, and hash each point to 1 or 0 according to which side of that plane it falls. Two points close together are usually on the same side; two far apart are often not. Repeat with many random directions and concatenate the bits, and the probability of a full collision falls smoothly with distance.

The elegance is that the guarantee is **probabilistic and tunable**: more bits means fewer false candidates and more misses, and you choose where to sit.

3.2 Storing points approximately

The second idea attacks memory rather than time.

A hundred million points in 128 dimensions, at four bytes per number, is about 51 GB — too much for one machine's memory, and memory is where speed comes from.

Product quantisation splits each vector into chunks, clusters the possible values of each chunk into a small codebook, and stores only which codebook entry each chunk is closest to. A 128-dimensional vector becomes, say, sixteen bytes. Distances are then computed approximately, from precomputed tables, without ever reconstructing the original vectors.

A factor of thirty in memory, at the cost of distances that are close to right rather than right. Given Section 2's bargain, that is a good exchange.

3.3 Navigable graphs

The idea behind most current libraries, and the one that repays understanding.

Build a graph in which each stored point is connected to some of its neighbours. To answer a query, start anywhere, look at the current node's neighbours, move to whichever is closest to the query, and repeat until no neighbour is closer. It is greedy hill-climbing on a graph.

Plain graphs of this kind get stuck in local minima and take many hops. The refinement — **hierarchical navigable small world** graphs — adds layers: a sparse top layer with long edges for covering ground quickly, denser layers below for refining. The search descends through the layers, and the number of hops grows roughly logarithmically with the number of points.

The structure is a deliberate echo of small-world networks: mostly local connections, plus a few long-range ones that collapse the diameter. The same principle that makes six degrees of separation work makes this search fast.

---

4. Where you will meet this

- **Recommendation.** "Customers who liked this also liked" is a nearest-neighbour query over item representations.
- **Deduplication.** Near-duplicate documents, images or records, at a scale where comparing every pair is quadratic and therefore impossible.
- **Reverse image and audio search.** Identifying a song from a few seconds is a nearest-neighbour lookup over acoustic fingerprints, and has been since long before deep learning.

- **Record linkage.** Matching customer records across systems that disagree about spelling.
- **Retrieval in database systems.** Several mainstream databases now offer a vector index as a first-class type, which is this machinery packaged.

The common shape: something turns items into points, and then everything interesting is a distance query. Lesson 8 covers where those points can come from without labels.

---

5. What to take from it

The curse of dimensionality is why this field exists. If distances stayed informative and trees kept pruning, exact search would be cheap and nobody would need any of Section 3.

Approximation is a design choice, not a failure. The systems above are not settling for less; they identified that the exactness they were paying for was not required, and spent the savings on scale.

Metric choice is a modelling decision. Euclidean distance and cosine similarity answer different questions, and every structure above is built for one metric. Choosing it is not a detail to be left to a default — the same point lesson 6 made about scaling, one level up.

And the k-NN caveat still applies, no matter how good the index: retrieving the nearest points quickly does not make "nearest" meaningful. An approximate search over five hundred uninformative dimensions returns wrong answers faster.

Where to look next

- **Indyk and Motwani (1998)** introduced locality-sensitive hashing; the original paper is readable and the intuition survives the notation.
- **Malkov and Yashunin**, on hierarchical navigable small world graphs — the method behind most current libraries.
- **Jégou, Douze and Schmid (2011)** on product quantisation.
- The documentation of **FAISS** and of **Annoy** is unusually good on the practical trade-offs, and both are open source and worth reading rather than only using.
- *Mining of Massive Datasets* (Leskovec, Rajaraman, Ullman), chapter 3, covers similarity search from first principles and is freely available.